The World Islamic Sciences and Education University

جامعة العلوم الإسلاميّة العالميّة

Faculty of Information Technology

كلّيّة تكنولوجيا المعلومات

Submitted by

Student name: Eman Awad Zuaiter

Student_No: 322601024

Student Email: eman.zuaiter7@gmail.com

Training Period

From 15th March 2026 to 3rd June 2026

Training Establishment: Royal Scientific Society (RSS)

70 Ahmad Al Tarawneh St, Al Jubaiha, Amman, Jordan

Phone: +962789202081

Acknowledgements

Sincere gratitude is expressed to Dr. Moath Tarawneh, my academic supervisor at the Faculty of Information Technology at WISE University, for the valuable guidance and ongoing support throughout this training project. The insightful advice and expertise provided have been instrumental in shaping this work.

Special appreciation is also extended to my supervisor at the Royal Scientific Society (RSS) for the professional mentorship and continuous encouragement during the three-month field training. Immersive experience within such a prestigious institution provided invaluable exposure to real-world deployment standards, advanced backend architectures, and rigorous system workflows. Deepest thanks are also due to the entire engineering team at RSS for welcoming me into their collaborative development environment, generously sharing their technical expertise, and guiding me through practical industry frameworks that significantly enhanced my engineering capabilities.

Furthermore, deep gratitude is felt toward my family and friends for their unwavering support and motivation throughout this graduation journey.

Abstract

This training report documents the design, implementation, and deployment of the ORYA Luxury Jewelry Store, developed as a primary capstone graduation project during the field training at the Royal Scientific Society (RSS). The platform bridges luxury e-commerce demands with modern software engineering by establishing a modular **Python Flask** framework and a responsive front-end visual language using **HTML, CSS and JS** variables and dynamic client-side translations. To enhance user experiences, ORYA integrates a highly sophisticated **agentic** Conversational AI Chatbot powered by **OpenAI GPT-4.1 mini** and a grounded **Retrieval-Augmented Generation (RAG**) pipeline utilizing **text-embedding-3-small** vector embeddings and a persistent **ChromaDB** database.

The system is divided into two primary target user categories: standard customer storefront accounts and authorized administrators. Customers enjoy isolated shopping cart drawers that prevent session leakage, real-time coupon code checkout validations, and a bilingual interface that switches layout directions (RTL/LTR) instantly. Store administrators manage inventory through Chart.js analytical sales reports, chronological order badges, and a custom multi-image product CRUD uploading module. Throughout the training period, a modern development methodology was adopted in which high-level system intent was communicated to AI coding assistants in natural language, allowing the developer to direct, review, and refine AI-generated implementations rather than authoring every line manually. This approach where human creativity defines what and AI accelerates the how significantly shortened development cycles, reduced repetitive coding overhead, and produced optimized, production-quality outputs at an unprecedented pace, reflecting an emerging paradigm where the developer evolves into an intelligent system architect and AI collaborator.

Table of Contents

Acknowledgements ..... **I**

Abstract ..... **II**

Table of Contents ..... Error! Bookmark not defined.

List of Tables ..... **VI**

List of Figure ..... **VV**

CHAPTER 1 INTRODUCTION & COMPANY PROFILE **1**

1.1 Field Training Introduction & System Vision 1

1.2 Training Establishment: Royal Scientific Society (RSS) 1

1.3 Internship Objectives & Technical Focus Areas 2

1.4 Scope of the ORYA Luxury Jewelry Store Project 3

1.5 Professional Growth & Technical Milestone Outlines 3

CHAPTER 2 TECHNICAL TRAINING TIMELINE & DEVELOPMENT METHODOLOGY **4**

2.1 Technical Training Phase 1: Core HTML5/CSS3 UI Structures 4

2.2 Technical Training Phase 2: JavaScript Interactivity & History Navigation 4

2.3 Technical Training Phase 3: Python Flask, Blueprints, & REST API Handlers 4

2.3.1 Secure Account Registration & Login Session Framework 6

2.4 Technical Training Phase 4: Relational SQLite Database Design 7

2.5 Technical Training Phase 5: NLP AI Conversational Chatbots & RAG Collections 8

CHAPTER 3 INTERACTIVE STOREFRONT USER EXPERIENCE & BILINGUAL UI **9**

3.1 Luxury Brand Identity & Visual CSS Tokens 9

3.2 Storefront Landing View Operations (index.html) 9

3.3 Comprehensive Collections Grid views (products.html) 10

3.4 Deep Parameter Spec Sheets & Care Guides (product.html) 11

3.5 Client-Side isolated Cart Drawer State Handling 11

3.6 Transactional checkout Forms & Promo Verification Engine 12

3.6.1 2 Two-Factor OTP Order Verification & Email Dispatcher 13

3.7 Sharing Wishlist & User Account Profiles (wishlist.html / account.html) 14

3.8 Bilingual Client-Side Translation Engines (i18n.js / lang.js) 15

CHAPTER 4 ADVANCED ADMIN MANAGEMENT DASHBOARD & OPERATIONS 16

4.1 Administrative Console Overview (dashboard.html) 16

4.2 Chart.js Sales Analytics & Bilingual Chart Rendering 16

4.3 Catalog CRUD Operations & Multi-Image Uploads (product_form.html) 17

4.4 Chronological Order processing Badges & Audit Trails 18

4.5 Global Settings, Coupon Configurations, & WhatsApp Channels 19

4.6 Dynamic Chatbot Memory Audits & Logs (chatbot_logs.html) 20

CHAPTER 5 RELATIONAL DATABASE SYSTEM SCHEMA & RELATIONSHIPS 21

5.1 Relational Schema Constraints & cascaded cascading deletions 21

5.2 SQLite Schema Columns and Key Declarations for all 11 Tables 21

CHAPTER 6 AI CHATBOT GROUNDED RAG & NLP AGENTICS **..... 28**

6.1 Conversational Chatbot NLP: GPT- 4.1 mini Model Integrations 28

6.2 Retrieval-Augmented Generation (RAG) Document parsing 28

6.3 text Chunking, overlapping (chunk size: 1000, overlap: 100) & text-embedding-3-small 29

6.4 Vector Store Management: ChromaDB Collections and persistence 29

6.5 Back-Office Database Tooling (Live Order Status, Return Ticket, Support Handoff) 29

List of Tables

Table 5.1: Users Relational Database Columns Specifications 31
Table 5.2: Products Relational Database Columns Specifications.....32
Table 5.3: Product Images Database Columns Specifications.....33
Table 5.4: Orders Database Columns Specifications.....33
Table 5.5: Order Items Database Columns Specifications.....34
Table 5.6: Coupons Database Columns Specifications.....34
Table 5.7: Site Settings Database Columns Specifications 35
Table 5.8: Announcements Database Columns Specifications.....35
Table 5.9: Chatbot Logs Database Columns Specifications.....36
Table 5.10: Return Requests Database Columns Specifications.....36
Table 5.11: Support Tickets Database Columns Specifications 36

List of Figure

Figure 1.1 : Royal Scientific Society (RSS) 2
Figure 2.2: ORYA Flask Backend Blueprint Modular Structural Map 5
Figure 2.3 : Apply Verification Code 7
Figure 2.4 : Login Interface 7
Figure 3.1 : orya Home Page 9
Figure 3.2 : Collections of orya Jewelry Store 10
Figure 3.3 : Storefront Product Grid Displaying Prices and Product Filtering Option 10
Figure 3.4 : Product Detail Page 11
Figure 3.5 : Shopping Cart 12
Figure 3.6 : Checkout Page 12
Figure 3.7 : Verification Interface 13
Figure 3.8 : Email With OTP Code 13
Figure 3.9 : Order Placement Success Message and Customer Confirmation Screen 14
Figure 3.10 : Wishlist sharing link module 14
Figure 4.1 : Admin Dashboard 16
Figure 4.2: Update Product and Primary Image 17
Figure 4.3: Update Product 17
Figure 4.4: Order processing Workspace 18
Figure 4.5: Some Setting 19
Figure 4.6: Add Coupon 19
Figure4.7: Mange Discounts 19
Figure 4.8: Chatbot History 20
Figure 6.1: orya Chatbot 28
Figure 6.2: Chatbot Routing 30
Figure 6.3: Search Products Tool 31
Figure 6.4: Create Purchase Request and OTP Tool 32
Figure 6.5: cancel order and human handoff tool 33
Figure 6.6: get order status tool 33
Figure 6.7: request return tool 34
Figure 6.8: admin notification triggered instantly via the human handoff tool 34

CHAPTER 1 INTRODUCTION & COMPANY PROFILE

1.1 Field Training Introduction & System Vision

Field training is a crucial academic pillar at the Faculty of Information Technology at WISE University. It bridges theoretical computer science lectures with real-world industry requirements. During the internship from March 15th to June 15th, 2022, the primary objective was to acquire hands-on experience in full-stack web engineering, API design, database schemas, and Natural Language Processing. The graduation capstone project developed during this training is the ORYA Luxury Jewelry Store. This system represents an elite, bilingual digital storefront tailored to the high-end jewelry market.

The vision behind the ORYA platform centers on absolute visual harmony, responsive efficiency, and robust security. Luxury buyers demand a digital storefront that functions with the same poise, exclusivity, and speed as an in-person boutique. In typical digital commerce platforms, product layouts can feel overcrowded, load times can disrupt user flow, and the shopping process is often interrupted by frequent page redirects. The ORYA platform addresses these constraints by implementing a single-page reactive shopping experience where all primary layouts load instantly, categories synchronize in a localized environment, and customer cart data is securely isolated by authenticated user sessions.

1.2 Training Establishment: Royal Scientific Society (RSS)

The Royal Scientific Society (RSS), founded in 1970 by HRH Prince El Hassan bin Talal, is Jordan's largest applied research institution. It is a non-governmental, not for profit organization dedicated to advancing science and technology for national development, located at 70 Ahmad Al Tarawneh St, Al Jubaiha, Amman, Jordan. RSS focuses on protecting human health, safeguarding the environment, and supporting sustainable economic growth. It operates specialized centers in environmental research, ICT, mechanical design, industrial chemistry, and quality assurance. With more than 500 staff, RSS provides testing, consultation, and certification services to industry, government, and academia. It also plays a vital role in shaping national policies through evidence-based recommendations and applied research. Regionally, RSS collaborates with universities, industries, and international organizations to promote innovation and knowledge transfer.

Within RSS, the Applied Artificial Intelligence Department is part of its ICT for Development programs. It harnesses machine learning, deep learning, and data science to address Jordan's pressing national challenges. In healthcare, AI supports diagnostics, medical imaging, and predictive analytics for better patient outcomes. In environmental research, AI models help manage water resources, monitor pollution, and forecast climate change impacts. For industry, AI enables predictive maintenance, quality assurance, and process optimization to boost operational efficiency. The department also advances digital transformation through natural language processing, cybersecurity, and intelligent data systems. Collaboration with Princess Sumaya University for Technology (PSUT) ensures the training of specialists and joint research projects that strengthen Jordan's AI ecosystem.

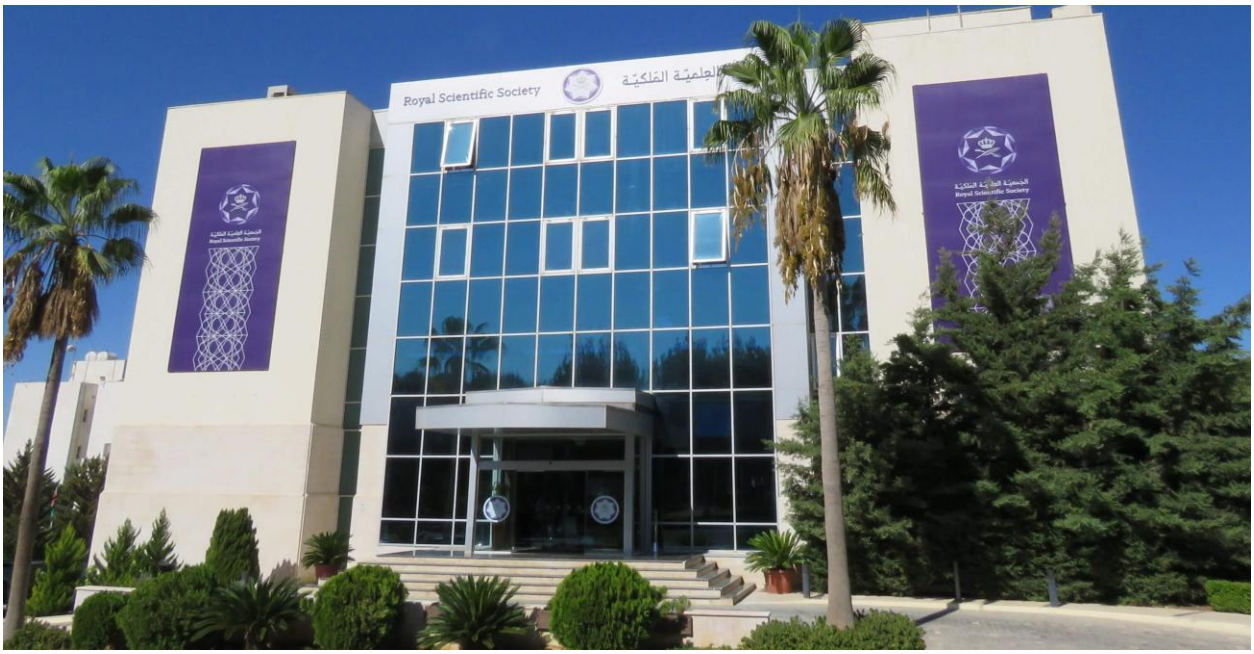


Figure 1.1 : Royal Scientific Society (RSS)

1.3 Internship Objectives & Technical Focus Areas

To satisfy the grading criteria of my academic supervisor and the professional standards of RSS, I established five key technical focus areas: Mastering frontend typography, glassmorphism overlays, and smooth CSS transitions. Developing modular **Flask** framework backends. Implementing secure, high-integrity **SQLite** relational database tables with cascades and foreign keys. Integrating **OpenAI GPT-4.1 mini** conversational NLP chatbot assistants with grounded **Retrieval-Augmented Generation (RAG)** collections.

1.4 Scope of the ORYA Luxury Jewelry Store Project

The scope of the ORYA platform encompasses both customer-facing storefront mhosting. back-office administrative portals. System boundaries separate unauthorized client guests, registered users, and authenticated store administrators through strict security layers, verifying that database edits, transaction reports, and system settings remain accessible only to authorized administrators.

1.5 Professional Growth & Technical Milestone Outlines

Throughout the training period, I progressed from basic frontend mockups to a highly optimized, production-ready bilingual web application. Weekly sprints focused on isolating complex technical systems, such as building client-side translation dictionaries, synchronizing analytical dashboards, managing vector embeddings, and resolving database bottlenecks. Working closely with my supervisors at RSS, I gained a deep appreciation for system stability, code maintainability, and user-centric design.

CHAPTER 2 TECHNICAL TRAINING TIMELINE & DEVELOPMENT METHODOLOGY

2.1 Technical Training Phase 1: Core HTML5/CSS3 UI Structures

The first month of training (March 15th to April 15th) focused on frontend styling and structural semantic layouts. I studied the important HTML tags provided in the university training syllabus, including <head> and <body> tags, structural block level containers, anchor <a> navigators, image <img> elements, and data <table> layouts. Rather than relying on simple styling, I learned to combine these tags with custom vanilla CSS3 variables to create rich visual alignments. I avoided generic CSS frameworks to maintain absolute control over the platform's luxury gold and slate color scheme.

2.2 Technical Training Phase 2: JavaScript Interactivity & History Navigation

The second phase of training focused on client-side interactivity using modern vanilla JavaScript. I built asynchronous cart drawers, **LocalStorage** synchronizers, and localized dictionary loaders. A major focus was history state management: I researched how to intercept default anchor click actions and push state updates via **history.pushState** on page changes. This allowed the platform to navigate between category collections instantly without full page reloads, satisfying the premium storefront speed criteria.

2.3 Technical Training Phase 3: Python Flask, Blueprints, & REST API Handlers

The third phase of training shifted to backend processing where, under the guidance of RSS supervisors, I studied Python Flask architectures and to avoid bloated monolithic files I implemented the Application Factory pattern with modularized routes using Blueprints such as auth.py for user credentials and Bcrypt session management, admin.py for administrative statistics, coupons, and orders APIs, products.py for storefront catalog displays and autocomplete searches, and chatbot.py for agentic NLP routes, and this clean separation ensures high maintainability and scalability of the system

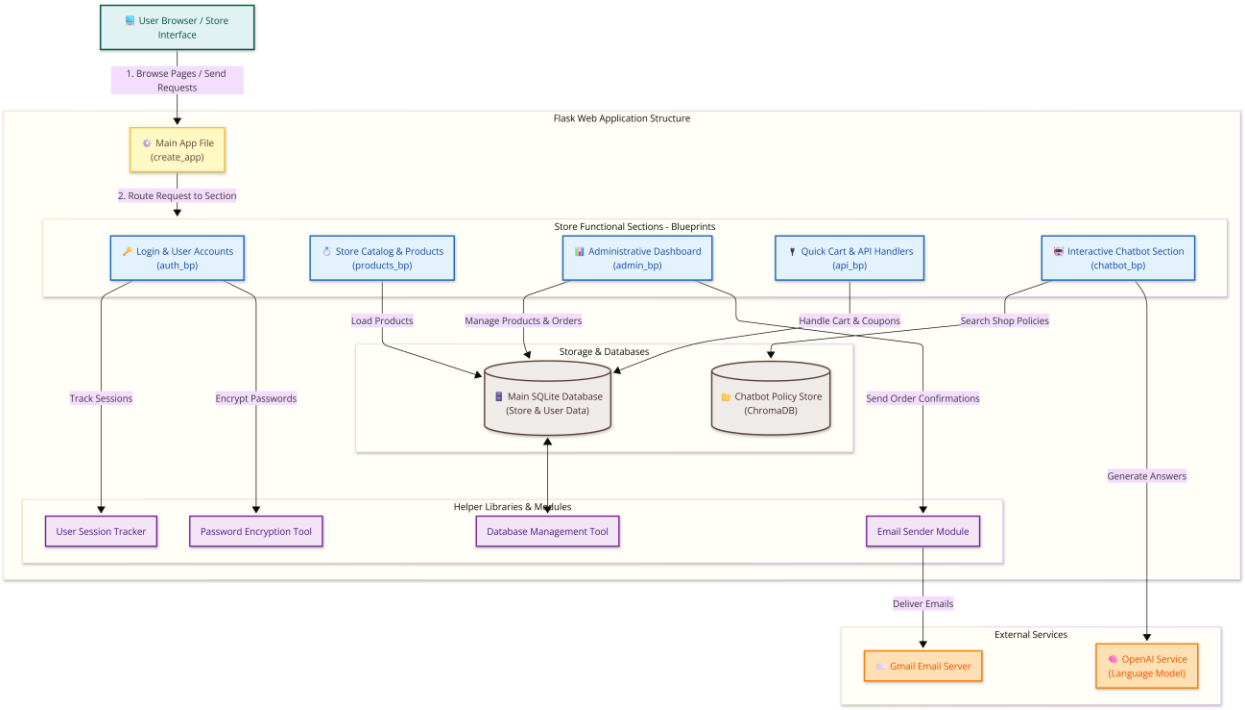


Figure 2.1: ORYA Flask Backend Blueprint Modular Structural Map

2.3.1 Secure Account Registration & Login Session Framework

To manage user access, I implemented a secure authentication system within the auth Blueprint. During user registration, inputs are validated using standard python libraries, such as the re library to enforce Jordanian mobile phone number formats (starting with 077, 078, or 079) and minimum password length constraints. To ensure verified signups, the system generates a 6-digit verification code (OTP) and dispatches it to the user's registered email using Flask-Mail. Account creation is finalized only after successful OTP validation, preventing fake registration entries. To guarantee secure storage, user passwords are encrypted using Flask-Bcrypt, which hashes passwords with salt, preventing database exposure. Session persistence and user logins are managed via Flask-Login using functions like login_user and logout_user alongside the login_required decorator, which secures user routes (such as wishlist management and order placement) against unauthorized access.

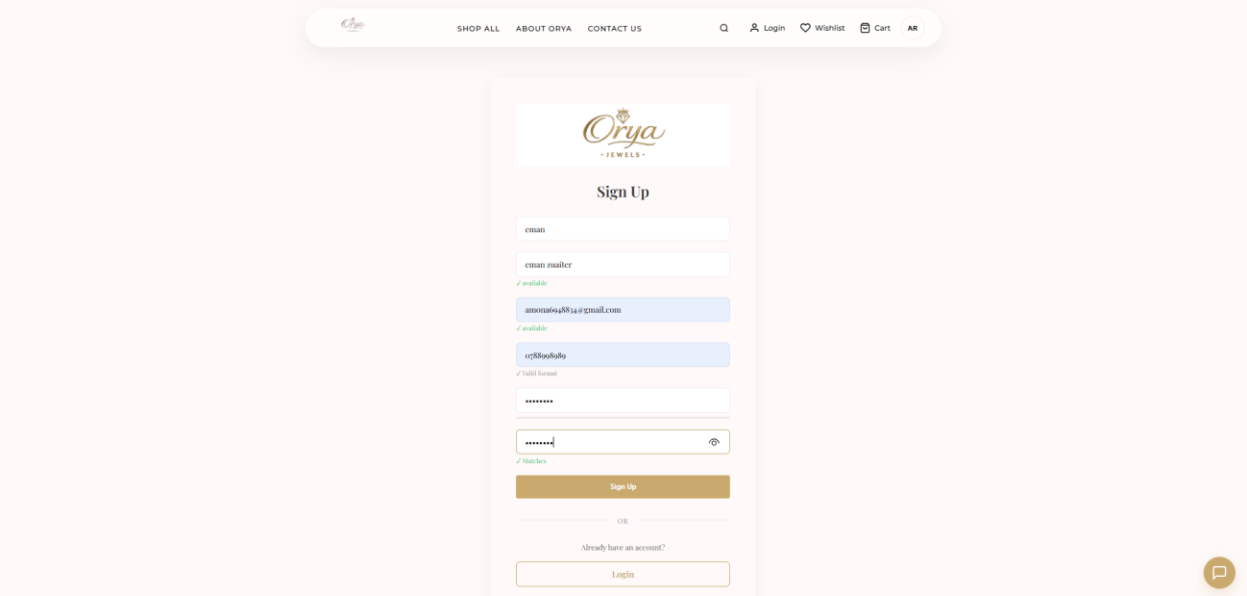


Figure 2.2: Sign Up Interface



Figure 2.3: Email With OPT Code

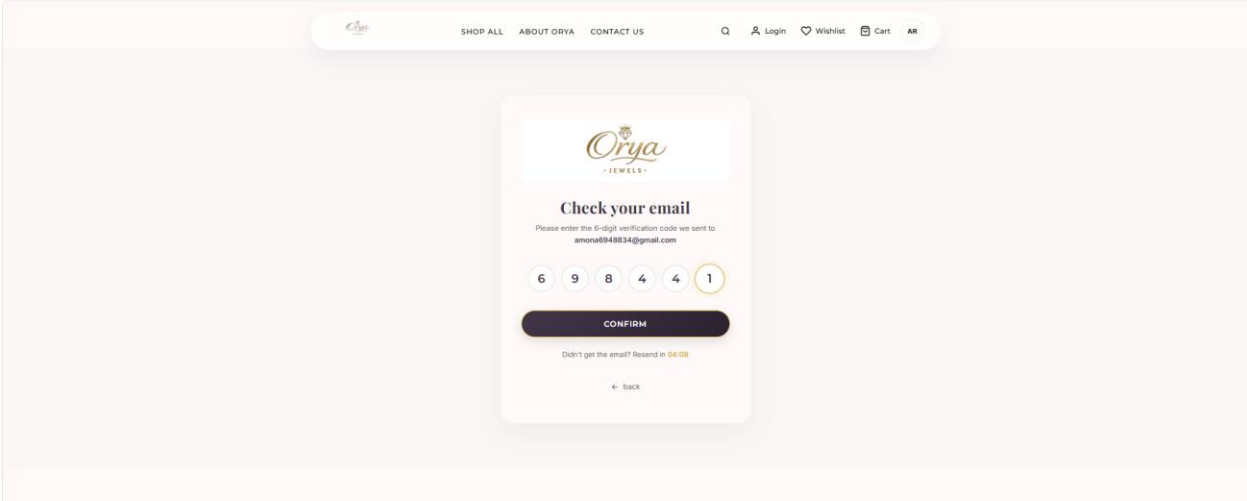


Figure 2.4: Apply Verification Code

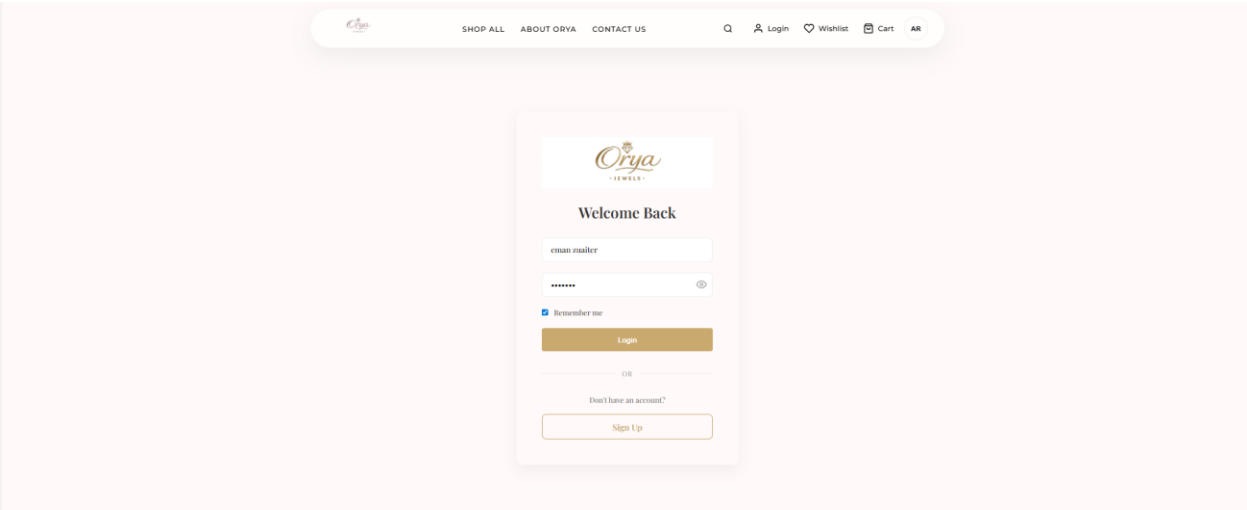


Figure 2.5 : Login Interface

2.4 Technical Training Phase 4: Relational SQLite Database Design

To persist the storefront catalog and user sessions, I designed a secure relational database using **SQLite** and **SQLAlchemy** ORM. This phase focused on defining relational integrity, unique constraints, indices, default values, and foreign key relations. Utilizing **SQLAlchemy** ORM allowed me to map models directly to **SQLite** tables, preventing raw SQL injections and ensuring type safety. Cascading delete hooks were configured across all relationships, ensuring database cleanliness when deleting products or users.

2.5 Technical Training Phase 5: NLP AI Conversational Chatbots & RAG Collections

The final phase of training merged software engineering with artificial intelligence. I studied Natural Language Processing (NLP) models and **Retrieval-Augmented Generation (RAG)** architecture. I integrated **OpenAI's GPT-4.1** mini model with LangChain's conversational agents, utilizing a persistent **vector store (ChromaDB)** to ground the bot in official store policy documents. I implemented unstructured document loaders, text chunk splitters, and OpenAI **text-embedding-3-small** vector encoders, building a highly performant and secure AI shopper.

CHAPTER 3 INTERACTIVE STOREFRONT USER EXPERIENCE & BILINGUAL UI

3.1 Luxury Brand Identity & Visual CSS Tokens

The visual identity of ORYA is crafted to present an elite, premium digital boutique. This is achieved by defining a structured set of CSS variables that govern the platform's color palette, typography, and spacing. The system avoids generic bright primary colors, using antique gold accents for interactive elements and deep slate and cream for backgrounds. Typography utilizes Playfair Display to convey elegance for headers, Montserrat for clean English text, and Cairo for beautifully shaped Arabic characters. Spacing, padding, and layout transitions are standardized, ensuring a consistent premium look across the entire application. The interface details align to visual luxury tokens.

3.2 Storefront Landing View Operations (index.html)

The storefront landing page (index.html) serves as the primary interactive entry point to the luxury boutique. It starts with a bilingual announcement banner displaying active promotions, while the navigation bar transitions to a frosted-glass effect upon scrolling. The hero section displays a full-bleed fine jewelry banner overlaid with elegant gold text and call-to-action buttons. Below this, interactive grids showcase the store's jewelry collections alongside a dynamically updated featured products display. Finally, a floating WhatsApp widget is anchored at the bottom to provide instant customer support. These combined features ensure an immersive and highly responsive user experience."

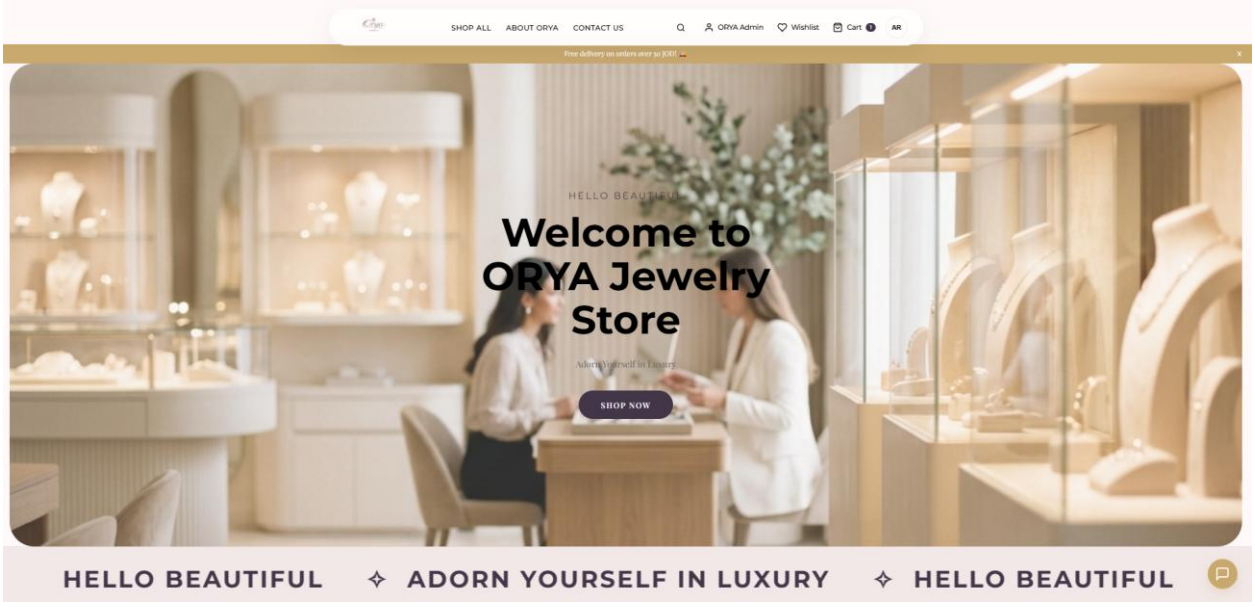


Figure 3.1 : orya Home Page

3.3 Comprehensive Collections Grid views (products.html)

The Collections page (products.html) displays jewelry categorized into six distinct grids. Each grid page features a custom collection header, a search input, and dynamic filters that sort products by price (low to high, high to low), material (gold, silver, plated), and stone type. Products are presented in elegant cards showing the product name, price, and thumbnail. Hovering over a card reveals secondary product images and a quick 'Add to Cart' button, encouraging user engagement. By utilizing modular templates, the collections load with high caching values.

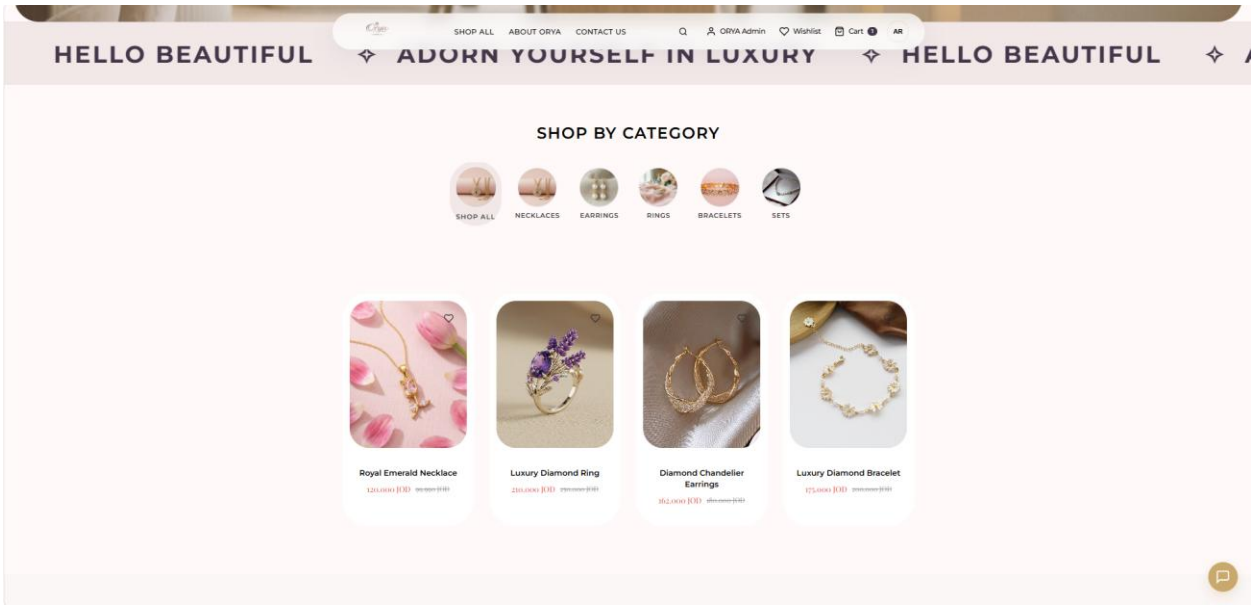


Figure 3.2 : Collections of orya Jewelry Store

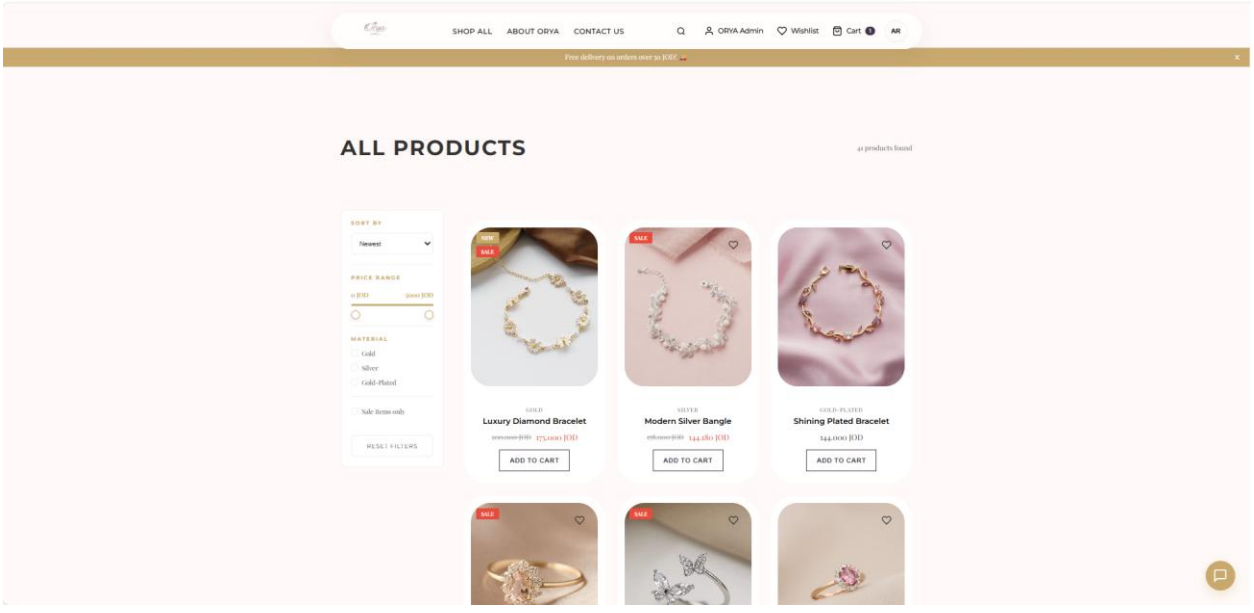


Figure 3.3 : Storefront Product Grid Displaying Prices and Product Filtering Option

3.4 Deep Parameter Spec Sheets & Care Guides (product.html)

The Product Detail view (product.html) features a split-screen design. One side displays an interactive image carousel that lets customers view the jewelry from multiple angles. The other side displays localized product details, pricing, and an interactive parameter specification sheet showing the exact material, length, ring sizing, gemstone details, and product weight. Tabbed containers provide customers with detailed care guides (e.g. keeping jewelry dry, avoiding perfumes) and shipping timelines, helping them make informed purchases.

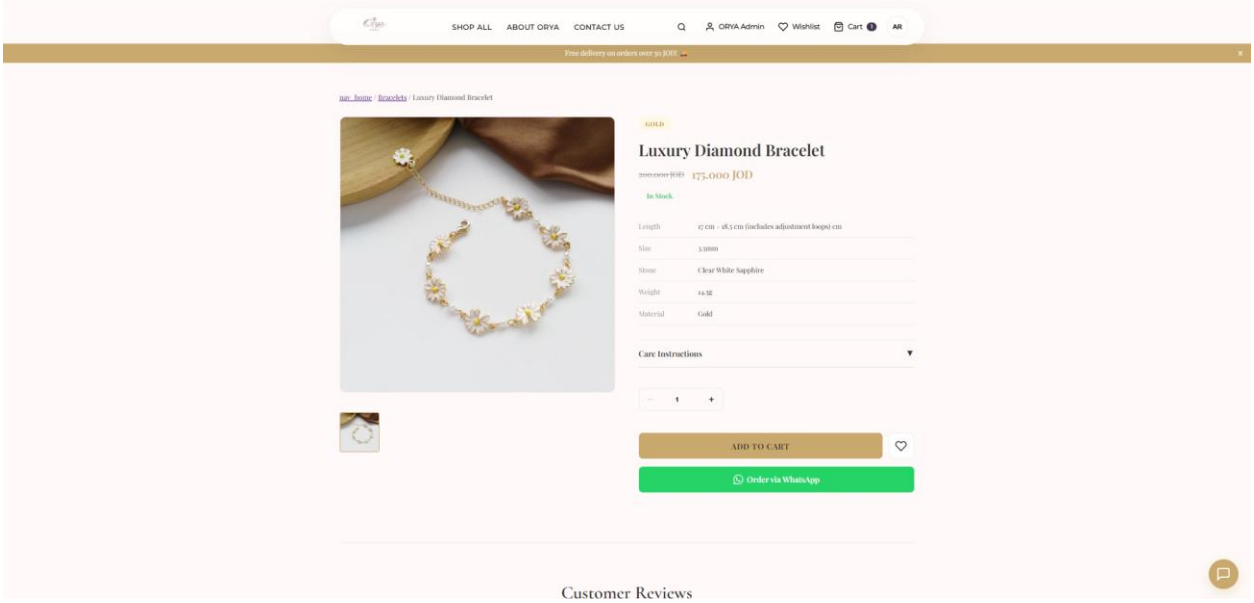


Figure 3.4 : Product Detail Page

3.5 Client-Side isolated Cart Drawer State Handling

To provide a seamless shopping process without disrupting the customer's navigation, the shopping cart is built as a slide-out drawer that transitions from the right or left of the viewport. Early cart systems often face synchronization issues where shopping carts persist across different user sessions on the same browser. To prevent this, ORYA implements a client-side cart isolation engine. The cart javascript reads the active user's ID: if the user is unauthenticated, cart arrays are saved under the **LocalStorage** key orya_cart_guest; if authenticated, the namespace switches to orya_cart_user_id. When logging out, the system isolates the keys, preventing guest users from accessing authenticated cart contents.

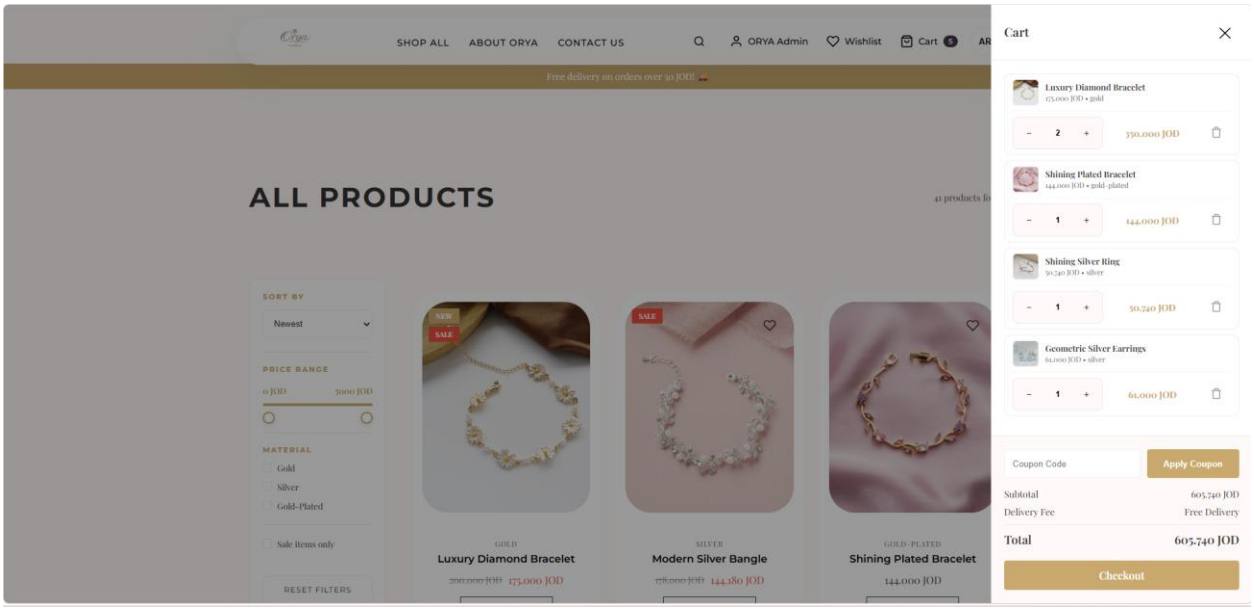


Figure 3.5 : Shopping Cart

3.6 Transactional checkout Forms & Promo Verification Engine

The Checkout page (checkout.html) simplifies the purchasing process into a clean form for shipping addresses, contact details, and promo codes. The pricing calculator operates with high mathematical precision: the subtotal is calculated, any valid coupon deductions are subtracted, the shipping fee is added, and the final total in Jordanian Dinars (JOD) is computed. Coupon validation happens dynamically on the server, auditing the coupon's active status, expiration date, and minimum order requirements before applying the discount to the checkout total.

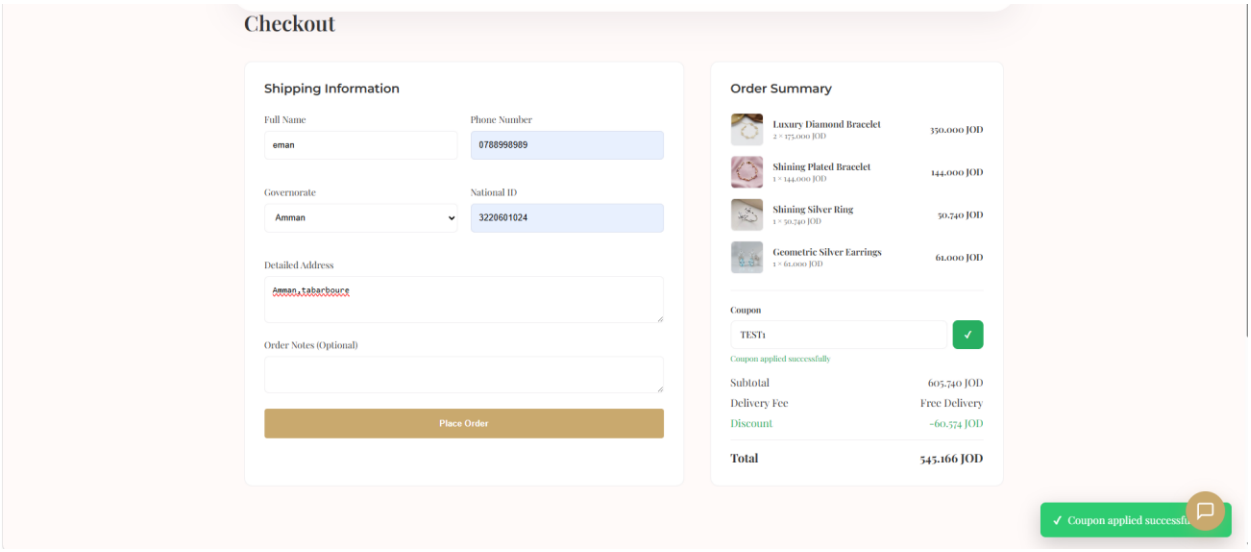


Figure 0.6 : Checkout Page

3.6.1 2 Two-Factor OTP Order Verification & Email Dispatcher

To mitigate fraudulent orders and verify customer identities during checkout, I developed a custom One-Time Password (OTP) verification engine. When a customer initiates checkout, the system calculates prices, decrements product stock quantities, and saves the transaction with a pending verification state. Simultaneously, the backend generates a random 6-digit verification code using Python's random library and records it in the secure session alongside a time timestamp. This OTP code is dynamically sent to the customer's verified email address using Flask-Mail connected to a Gmail SMTP server. The order status is updated to be confirmed only after the customer submits the matching code via the verify-order-otp route within a 5-minute expiration window, ensuring strong transactional integrity.

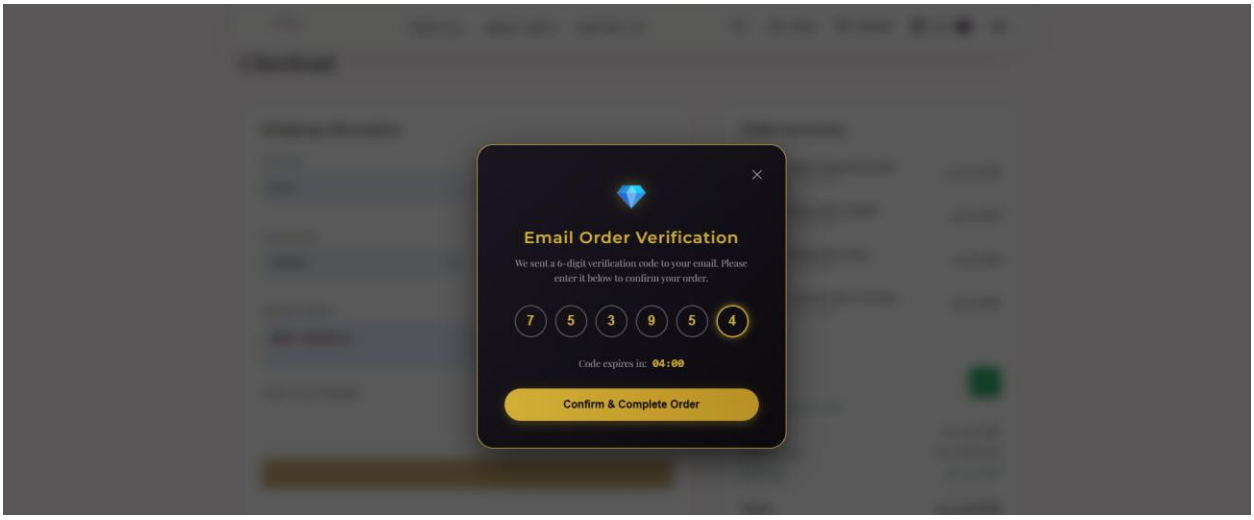


Figure 3.7 : Verification Interface



Figure 3.8 : Email With OTP Code

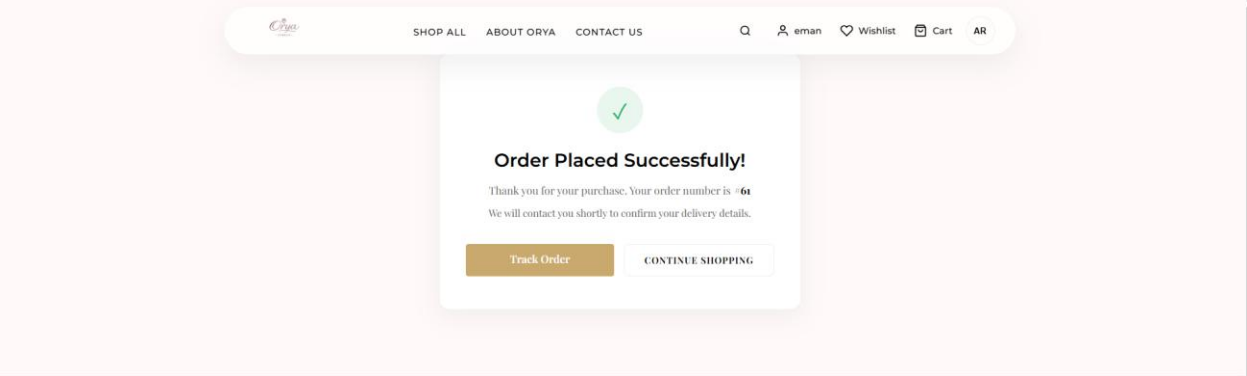


Figure 3.9 : Order Placement Success Message and Customer Confirmation Screen

3.7 Sharing Wishlist & User Account Profiles (wishlist.html / account.html)

The Wishlist page (wishlist.html) allows customers to save their favorite jewelry pieces. Customers can toggle a heart icon on product cards to add or remove items dynamically. A unique feature is the 'Share Wishlist' button, which generates a copyable link that lets family and friends view the customer's curated jewelry collection. The Customer Account Portal (account.html) allows registered users to update their profiles, manage delivery addresses, and view historic purchase receipts and tracking statuses.

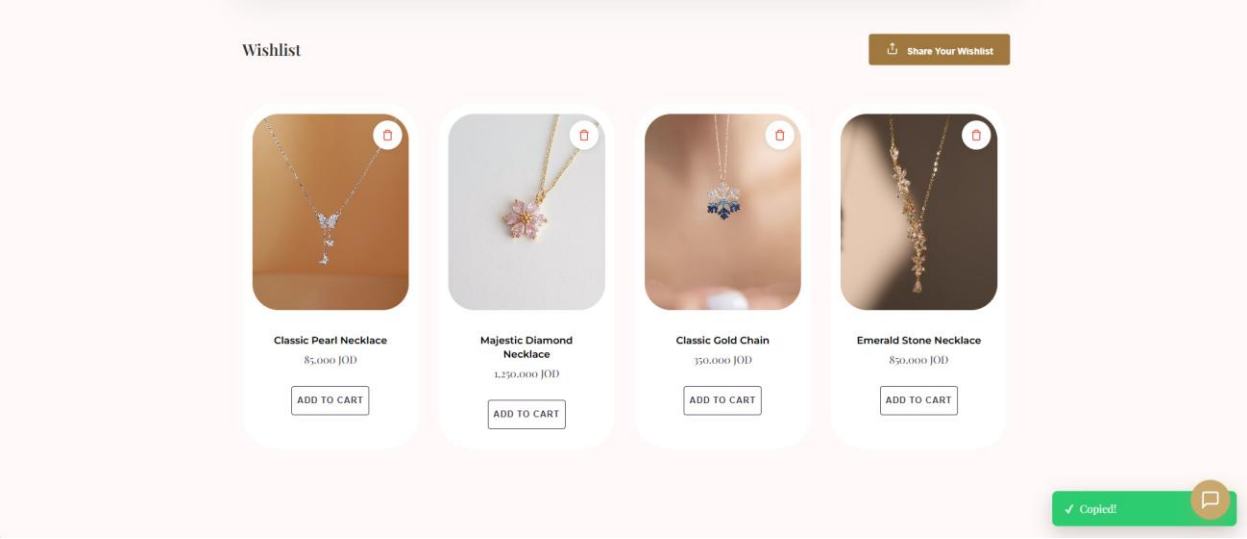


Figure 0.10 : Wishlist sharing link module

3.8 Bilingual Client-Side Translation Engines (i18n.js / lang.js)

To provide a high-end bilingual shopping experience without server-side loading delays, ORYA implements a client-side localization system. The engine consists of two core files: i18n.js (which holds a comprehensive translation dictionary for UI elements, labels, buttons, and alert messages in Arabic and English) and lang.js (which contains functions to get, set, and toggle the active language state). Active language is saved in the browser session, allowing immediate language changes across the page.

CHAPTER 4 ADVANCED ADMIN MANAGEMENT DASHBOARD & OPERATIONS

4.1 Administrative Console Overview (dashboard.html)

The ORYA Admin Dashboard (dashboard.html) is a secure portal designed for store managers. The interface features a dark side-navigation menu and a white content workspace. The home view displays key performance indicators (KPIs) in clean cards: Total Revenue, Pending Orders, Successful Deliveries, Active Coupons, and Registered Customers. Managers can click on any card to view detailed historical logs, enabling rapid business audits. We ensure role permissions strictly separate administrators and general storefront users.

4.2 Chart.js Sales Analytics & Bilingual Chart Rendering

To visualize sales performance, the dashboard integrates interactive charts powered by Chart.js. The charts query backend APIs to display weekly revenue, category sales distributions, and order status summaries. The charts are fully integrated with the storefront's bilingual engine: toggling between Arabic and English updates all chart labels, axes, and legends dynamically without page reloads, ensuring a consistent user experience.

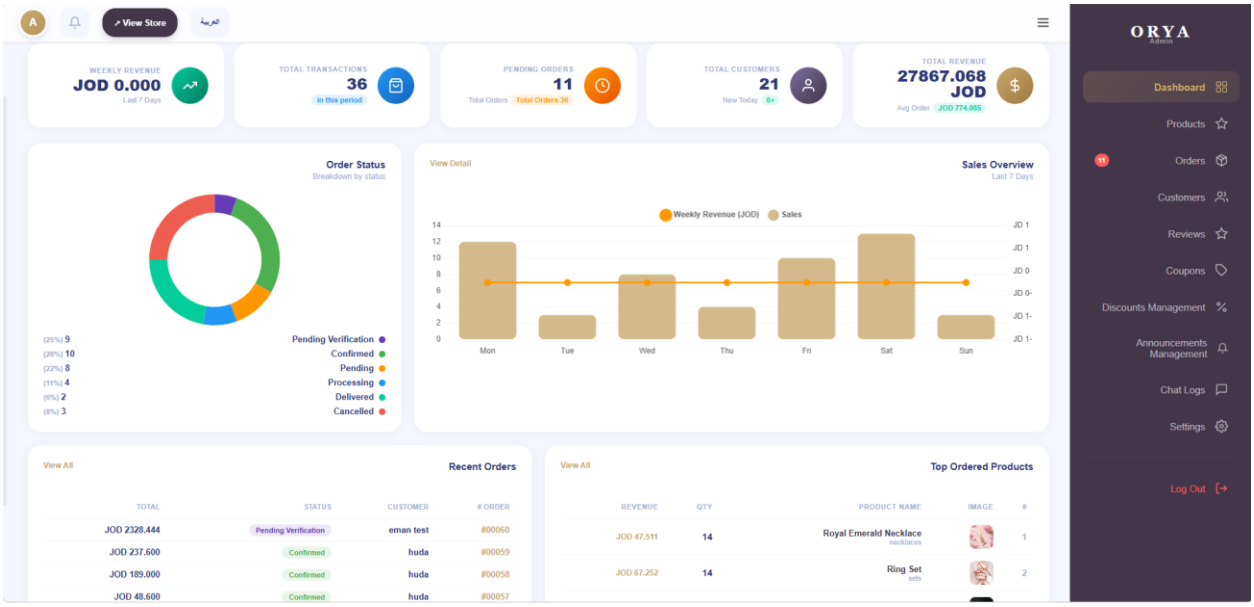


Figure 4.1 : Admin Dashboard

4.3 Catalog CRUD Operations & Multi-Image Uploads (product_form.html)

The Product Manager (products.html and product_form.html) provides administrators with complete control over the store's inventory. The product creation form includes validated inputs for names, prices, categories, weights, and materials. The image upload system supports adding multiple product images: administrators can paste direct image URLs, which are validated for secure HTTP/HTTPS protocols, or drag and drop local files. An AJAX script uploads files to static/images/products with unique filenames. Uploaded images are displayed as thumbnails; administrators can drag thumbnails to change their display order or click a thumbnail to mark it as the Primary Image.

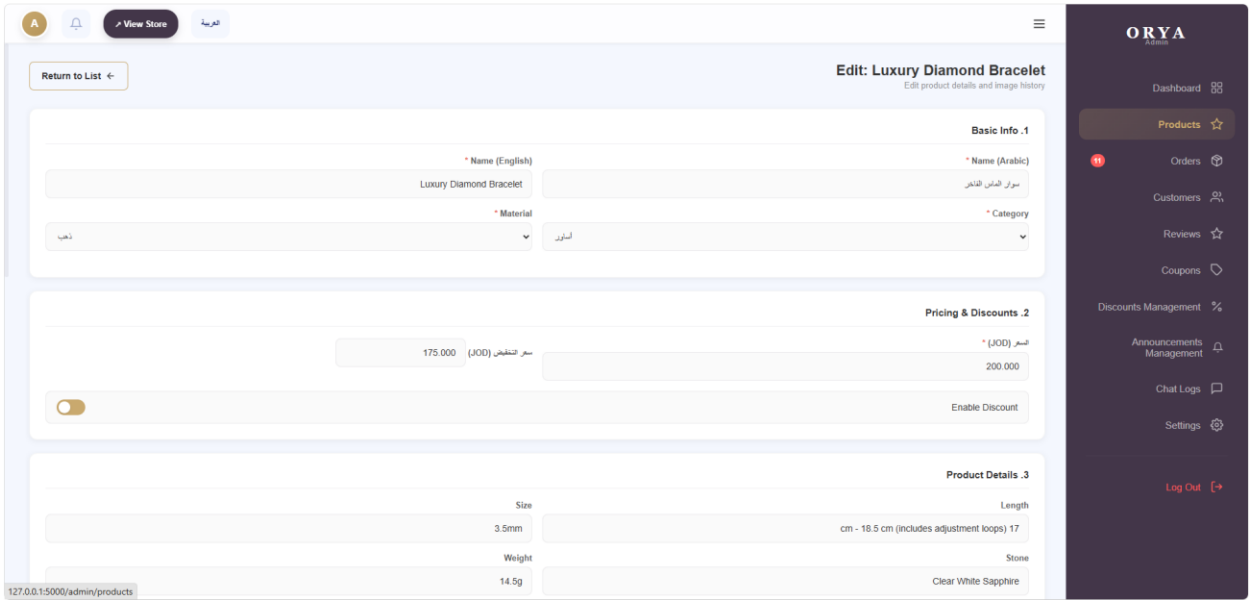


Figure 4.3: Update Product

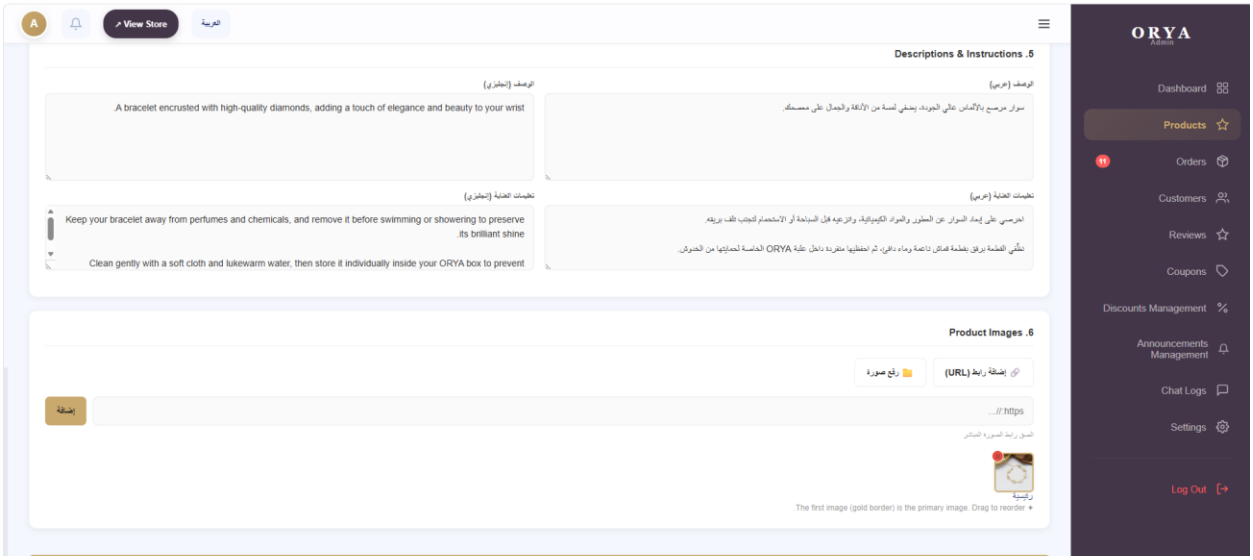


Figure 4.2: Update Product and Primary Image

4.4 Chronological Order processing Badges & Audit Trails

The Order processing workspace lists customer purchases chronologically, displaying the order ID, name, date, item count, total price, and active status. Status-colored badges represent the order's phase: Pending (Yellow), Processing (Blue), Delivered (Green), and Cancelled (Red). Administrators can click an order to view the detail page, edit delivery coordinates, write administrative notes, and update order statuses, which instantly triggers transactional update logs.

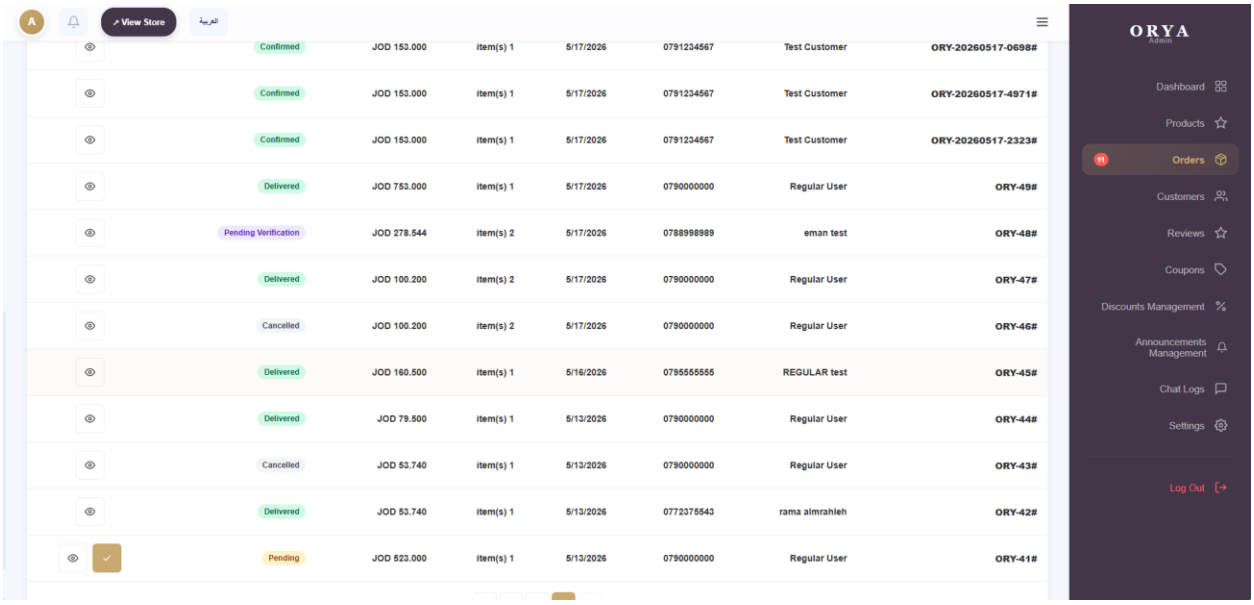


Figure 4.4: Order processing Workspace

4.5 Global Settings, Coupon Configurations, & WhatsApp Channels

The Settings view provides store managers with control over global operations where the Coupon Editor allows them to create coupons, define deduction types as either percentage or fixed JOD, set usage limits, and configure expiration dates, while the global configurations section enables managers to modify support WhatsApp numbers, shipping fees, and free delivery thresholds, and the Customer Database

presented in customers.html displays registered customers giving managers the ability to inspect historical purchases and review support requests.

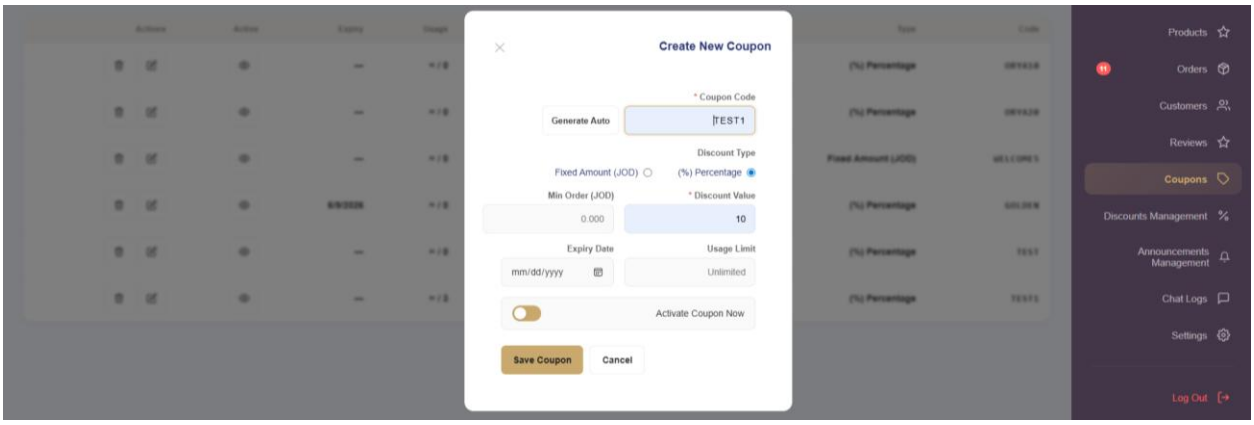


Figure 4.5: Add Coupon

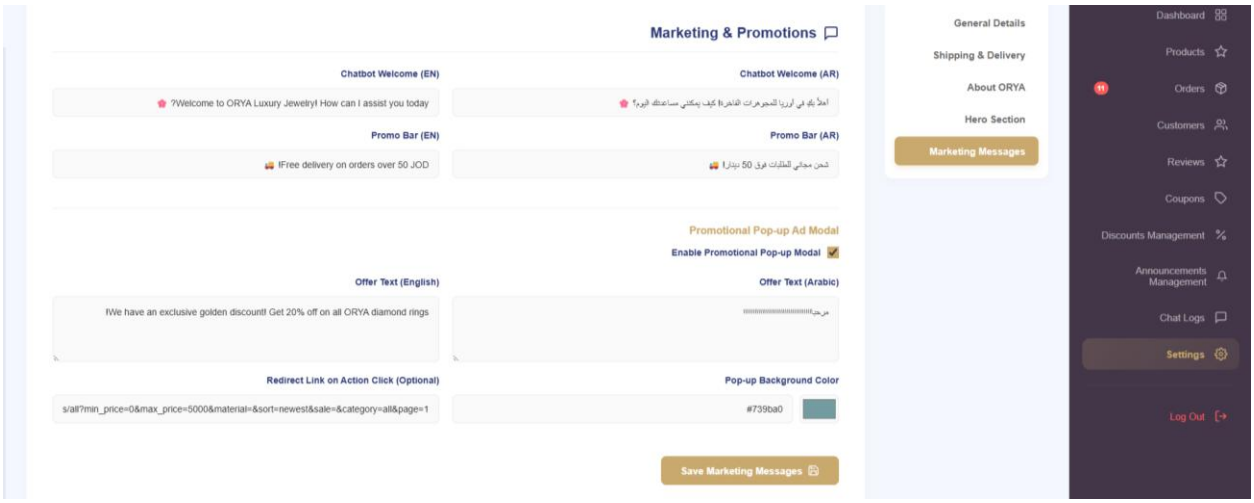


Figure 4.6: Some Setting

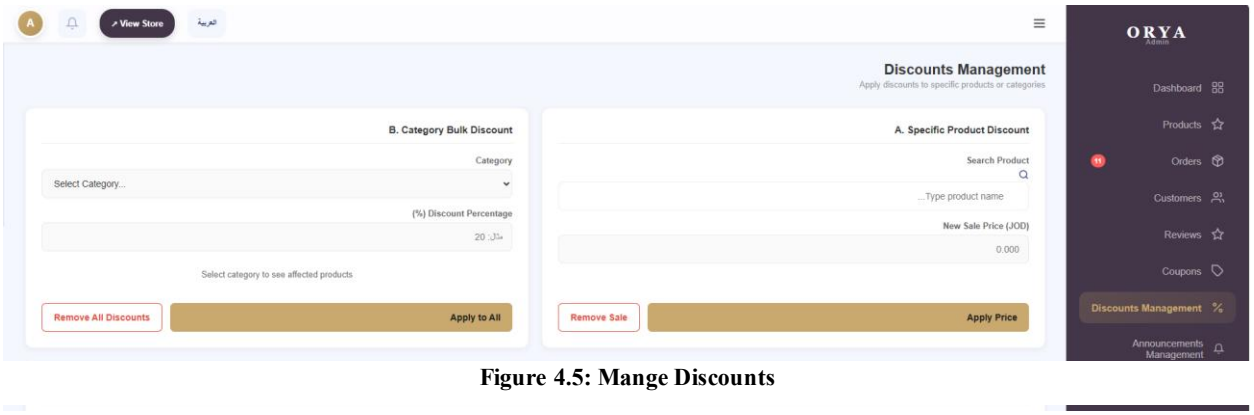


Figure 4.5: Mange Discounts

4.6 Dynamic Chatbot Memory Audits & Logs (chatbot_logs.html)

The Chatbot Interaction Log (chatbot_logs.html) is a specialized administrative view that tracks conversation histories between the AI assistant and storefront visitors. Administrators can audit session IDs, inspect the exact user queries and generated replies, track the language used (Arabic/English), and view **RAG** retrieval scores. This log helps the management team analyze customer questions, identify catalog gaps, and refine the AI chatbot's responses over time.

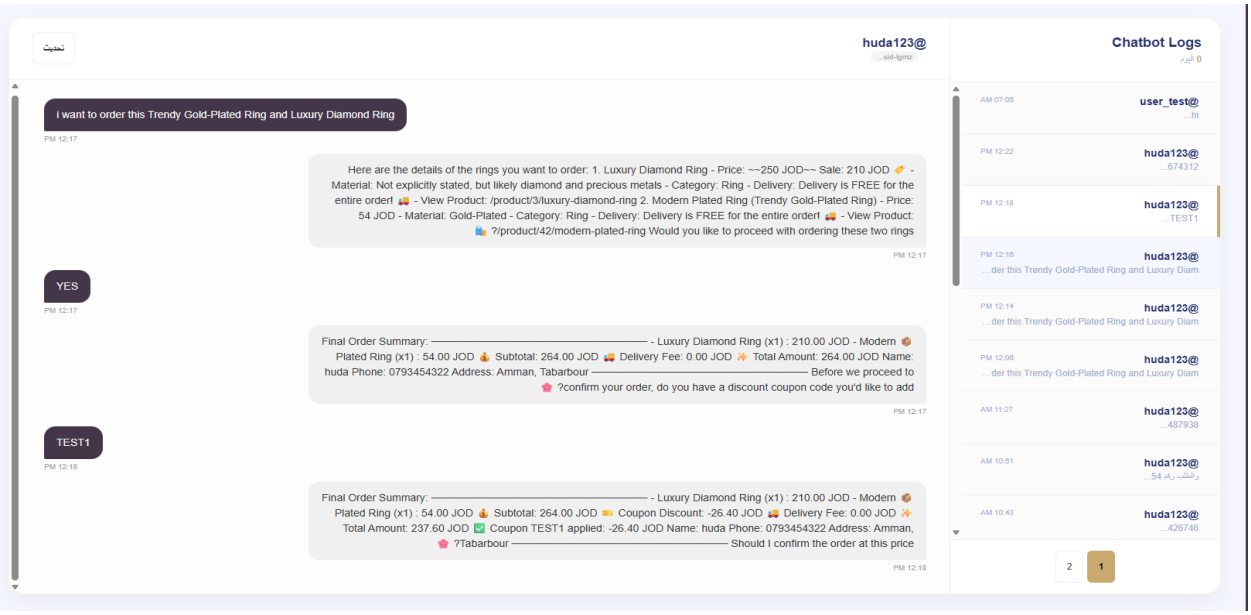


Figure 4.6: Chatbot History

CHAPTER 5 RELATIONAL DATABASE SYSTEM SCHEMA & RELATIONSHIPS

5.1 Relational Schema Constraints & cascaded cascading deletions

The ORYA relational database is managed by **SQLAlchemy** ORM and persisted in **SQLite** at instance/orya.db. The database is designed with strong relational integrity: the User model maintains one-to-many relationships with orders, reviews, and Wishlist; the Product model connects to multiple Product Image records with cascading delete constraints, ensuring that removing a product automatically cleans up its associated images. This relational schema provides high data consistency and security.

5.2 SQLite Schema Columns and Key Declarations for all 11 Tables

To ensure system stability, every database column is strictly configured with type declarations, uniqueness constraints, and default values. Below is the complete relational specification for the 11 active tables:

Table 5.1: users (User Model Columns Specifications)

Column Name	Data Type	Constraints	Description
id	Integer	Primary Key	Unique identifier for each user record.
username	String (50)	Unique, Nullable=False	The unique username used for user login.
full_name	String (120)	Nullable=False	The full legal name of the user.
email	String (120)	Unique, Nullable=False	The validated email address of the user.
phone	String (20)	Nullable=False	The contact phone number used for delivery.
password_hash	String(255)	Nullable=False	Securely hashed user password using Bcrypt.
role	String(20)	Default='user'	Determines access level: user, admin, super_admin.
is_active	Boolean	Default=True	Indicates if the account is currently active.
created_at	DateTime	Default=datetime.utcnow	Timestamp when the user account was created.

Table 5.2: products (Products Model Columns Specifications)

Column Name	Data Type	Constraints	Description
id	Integer	Primary Key	Unique identifier for the product.
name_ar	String (200)	Nullable=False	Arabic name of the jewelry item.
name_en	String (200)	Nullable=False	English name of the jewelry item.
category	String (50)	Nullable=False	Category: necklaces, earrings, rings, bracelets, sets.
price_jod	Numeric (10, 3)	Nullable=False	The original retail price in Jordan Dinars.
sale_price_jod	Numeric (10, 3)	Nullable=True	The promotional sale price, if applicable.
is_on_sale	Boolean	Default=False	Flag indicating if the product has a discount active.
material	String (50)	Nullable=False	Material composition: gold, silver, plated.
length	String (50)	Nullable=True	Length specification for necklaces.
size	String (50)	Nullable=True	Size parameters for rings.
stone	String (100)	Nullable=True	Gemstone details (e.g. Cubic Zirconia).
weight	String (50)	Nullable=True	Weight of the piece (e.g. 4.5g).
description_ar	Text	Nullable=True	Detailed Arabic description.
description_en	Text	Nullable=True	Detailed English description.
care_ar	Text	Nullable=True	Arabic care instructions.
care_en	Text	Nullable=True	English care instructions.
stock_qty	Integer	Default=0	Available stock units in inventory.
is_new	Boolean	Default=False	Flag to display 'New' badge.
is_visible	Boolean	Default=True	Controls visibility to customers.
sales_count	Integer	Default=0	Total units sold, updated on checkout.
created_at	DateTime	Default=datetime.utcnow	Timestamp when product was added.

Table 5.3: product_images (Product Images Database Columns)

Column Name	Data Type	Constraints	Description
id	Integer	Primary Key	Unique identifier for the image record.
product_id	Integer	ForeignKey('products.id')	Links the image to its parent product.
image_path	String(500)	Nullable=False	Local directory path or external image URL.
is_primary	Boolean	Default=False	Indicates if this image is the primary display image.
sort_order	Integer	Default=0	Determines thumbnail display order in lists.

Table 5.4: orders (Orders Database Columns Specifications)

Column Name	Data Type	Constraints	Description
id	Integer	Primary Key	Unique identifier for the order record.
user_id	Integer	ForeignKey('users.id')	Links the order to the purchasing user account.
status	String (20)	Default='pending'	Order stage: pending, processing, delivered, cancelled.
subtotal_jod	Numeric (10, 3)	Nullable=False	Sum of all order items prior to deductions.
discount_jod	Numeric (10, 3)	Default=0	Total savings applied from coupons.
delivery_fee_jod	Numeric (10, 3)	Default=0	The cost of shipping calculated for the order.
total_jod	Numeric (10, 3)	Nullable=False	The final transaction amount.
coupon_code	String (50)	Nullable=True	The promo code applied to this purchase.
payment_method	String (50)	Default='cash_on_delivey	Payment type choice.
full_name	String (120)	Nullable=False	Full delivery contact name.
phone	String (20)	Nullable=False	The contact phone number for delivery.
city	String (100)	Nullable=False	The city for shipping.
area	String (100)	Nullable=False	The neighborhood or area for delivery.
address	Text	Nullable=False	Complete street address details.

national_id	String (20)	Nullable=True	Optional verification ID required for shipping.
notes	Text	Nullable=True	Special customer shipping instructions.
admin_notes	Text	Nullable=True	Private internal order notes by admins.
created_at	DateTime	Default=datetime.utcnow	Timestamp when the order was placed.
updated_at	DateTime	Default=datetime.utcnow	Timestamp of the last status update.

Table 5.5: order_items (Order Items Database Columns)

Column Name	Data Type	Constraints	Description
id	Integer	Primary Key	Unique identifier for the order item.
order_id	Integer	ForeignKey('orders.id')	Links the item to its parent order.
product_id	Integer	ForeignKey('products.id')	Links the item to the product purchased.
quantity	Integer	Nullable=False	Total quantity of the product ordered.
unit_price_jod	Numeric (10, 3)	Nullable=False	The actual unit price locked at time of purchase.

Table 5.6: coupons (Coupons Database Columns Specifications)

Column Name	Data Type	Constraints	Description
id	Integer	Primary Key	Unique identifier for the coupon.
code	String(50)	Unique, Nullable=False	The alphanumeric code typed by customers.
type	String(20)	Nullable=False	Deduction format: percentage or fixed JOD.
value	Numeric(10, 3)	Nullable=False	Deduction value.
usage_limit	Integer	Nullable=True	Max times this coupon can be used globally.
used_count	Integer	Default=0	Active counter of successful redemptions.
min_order_jod	Numeric(10, 3)	Default=0	Min subtotal needed to activate the coupon.
is_active	Boolean	Default=True	Controls if the coupon is currently valid.

expires_at	DateTime	Nullable=True	The expiration date and time of the coupon.
created_at	DateTime	Default=datetime.utcnow	Timestamp when the coupon was created.

Table 5.7: site_settings (Site Settings Database Columns)

Column Name	Data Type	Constraints	Description
id	Integer	Primary Key	Unique setting identifier.
key	String (100)	Unique, Nullable=False	The setting lookup key (e.g., delivery_fee_jod).
value_ar	Text	Nullable=True	The Arabic setting value or rule configuration.
value_en	Text	Nullable=True	The English setting value or rule configuration.

Table 5.8: announcements (Announcements Database Columns)

Column Name	Data Type	Constraints	Description
id	Integer	Primary Key	Unique identifier for the announcement banner.
text_ar	String (500)	Nullable=False	Arabic promotional text displayed in the header banner.
text_en	String (500)	Nullable=False	English promotional text displayed in the header banner.
is_active	Boolean	Default=False	Controls if the banner is currently displayed.
bg_color	String (20)	Default='#C9A96E'	Hex code color background for the banner.
link	String (500)	Nullable=True	Optional redirect URL when clicking the banner.
created_at	DateTime	Default=datetime.utcnow	Timestamp when the announcement was created.

Table 5.9: chatbot_logs (Chatbot Logs Database Columns)

Column Name	Data Type	Constraints	Description
id	Integer	Primary Key	Unique identifier for each log entry.
user_id	Integer	ForeignKey('users.id')	Associated user account (nullable for guest chats).
session_id	String(100)	Nullable=False	Unique session ID generated per chat conversation.
user_message	Text	Nullable=False	The exact message sent by the user.
bot_reply	Text	Nullable=False	The response generated by the AI Chatbot.
lang	String (5)	Default='ar'	Language used for the conversation: ar or en.
created_at	DateTime	Default=datetime.utcnow	Timestamp of the interaction.

Table 5.10: return_requests (Return Requests Database Columns)

Column Name	Data Type	Constraints	Description
id	Integer	Primary Key	Unique ticket identifier.
order_id	Integer	ForeignKey('orders.id')	The associated order the customer wants to return.
user_id	Integer	ForeignKey('users.id')	Associated user account initiating the return.
reason	Text	Nullable=False	Customer's detailed explanation for returning items.
status	String (20)	Default='pending'	Ticket status: pending, contacted, resolved.
created_at	DateTime	Default=datetime.utcnow	Timestamp when the return request was created.

Table 5.11: support_tickets (Support Tickets Database Columns)

Column Name	Data Type	Constraints	Description
id	Integer	Primary Key	Unique ticket identifier.
user_id	Integer	ForeignKey('users.id')	Associated user account (nullable for guests).
session_id	String (100)	Nullable=True	Associated chat session ID if created via the chatbot.
description	Text	Nullable=False	Detailed support issue description.
status	String (20)	Default='open'	Ticket status: open, closed.
created_at	DateTime	Default=datetime.utcnow	Timestamp when the ticket was created.

CHAPTER 6 AI CHATBOT GROUNDED RAG & NLP AGENTICS

6.1 Conversational Chatbot NLP: GPT- 4.1 mini Model Integrations

To provide immediate customer support without human latency, ORYA integrates an intelligent chatbot powered by OpenAI's GPT-4.1 mini model. While standard chatbots rely on hardcoded rules, ORYA utilizes a **Retrieval-Augmented Generation (RAG)** pipeline. This system connects the language model to verified store documentation, allowing the chatbot to answer questions about shipping times, refund policies, and product details with high accuracy, preventing common hallucination issues.

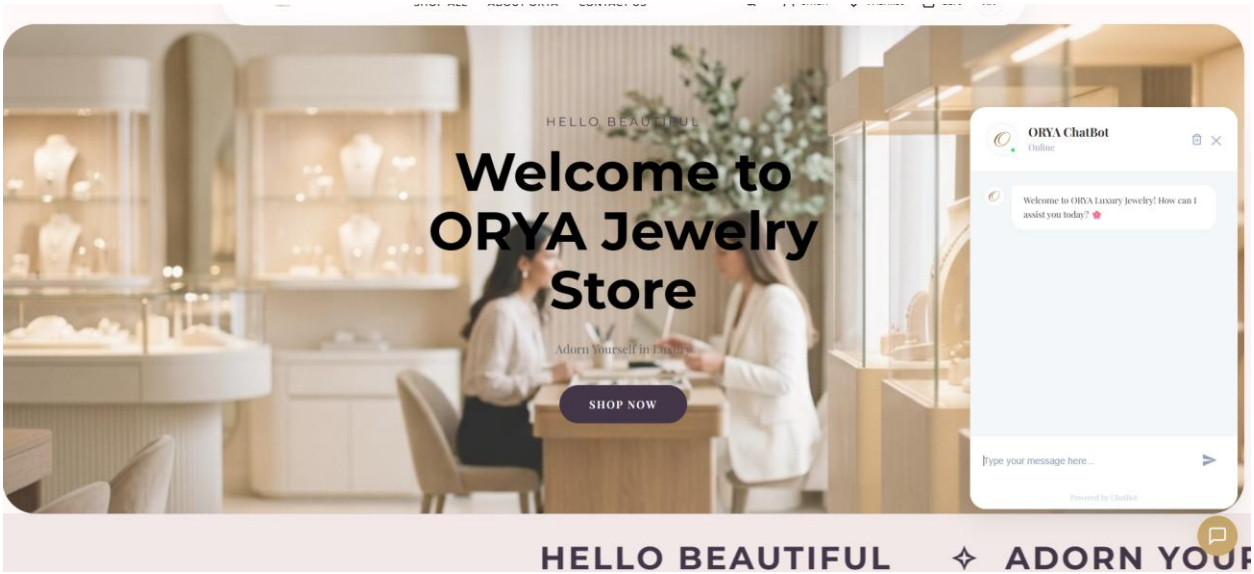


Figure 6.1: orya Chatbot

6.2 Retrieval-Augmented Generation (RAG) Document parsing

The RAG parsing architecture handles unstructured information by loading text guidelines using specialized file parser classes (specifically Text Loader for plain text policies and catalog parameters). Once parsed, the raw document strings are normalized using clean character-decoding filters, preparing the data for the recursive text splitting algorithm.

6.3 text Chunking, overlapping (chunk size: 1000, overlap: 100) & text-embedding-3-small

To optimize retrieval accuracy, documents are split into cohesive text blocks using strict **chunking** parameters where the text is divided into chunks of 1000 characters using the **RecursiveCharacterTextSplitter**. To maintain context across block boundaries, each chunk overlaps the previous one by 100 characters. These text chunks are then processed through OpenAI's **text-embedding-3-small embedding model**, generating 1536-dimensional vectors that are persisted in the vector store, where this precise chunk size and **chunk overlap** minimizes border information loss

6.4 Vector Store Management: ChromaDB Collections and persistence

Vector embeddings are stored and managed inside a high-performance vector database (ChromaDB) located in the app/chroma_db directory. This vector store enables semantic search, allowing the system to retrieve information based on conceptual meaning rather than exact keyword matches. To optimize search queries, the system separates information into two distinct vector collections, where the orya_knowledge collection stores static policy files while the orya_products collection indexes the active product catalog. This separation prevents search conflicts, ensuring that product searches and policy questions query the appropriate collections. ChromaDB offers high speed index calculations and similarity matching.

6.5 Back-Office Database Tooling (Live Order Status, Return Ticket, Support Handoff)

To extend the chatbot beyond simple question-and-answering capabilities, the agent is equipped with a suite of custom database tools that enable it to perform secure backend actions. Specifically, the **search_products_tool** allows the chatbot to scan the jewelry catalog using semantic similarity to match customer preferences, while the **get_order_status_tool** queries the SQLite database to return real-time tracking information. Furthermore, the chatbot can manage orders directly using the **cancel_order_tool** to cancel pending orders and the **get_promotions_tool** to retrieve active storefront coupons and promotional announcements. For post-purchase operations, the **request_return_tool** registers item returns requests into the database, and the **human_handoff_tool** logs complex support tickets to alert human administrators. Finally, the agent facilitates purchases via the **create_purchase_request_tool** to construct new orders and the **send_order_otp_tool** to trigger verification emails. This establishes a fully agentic workflow where the GPT-4.1 mini agent autonomously makes database dedications.

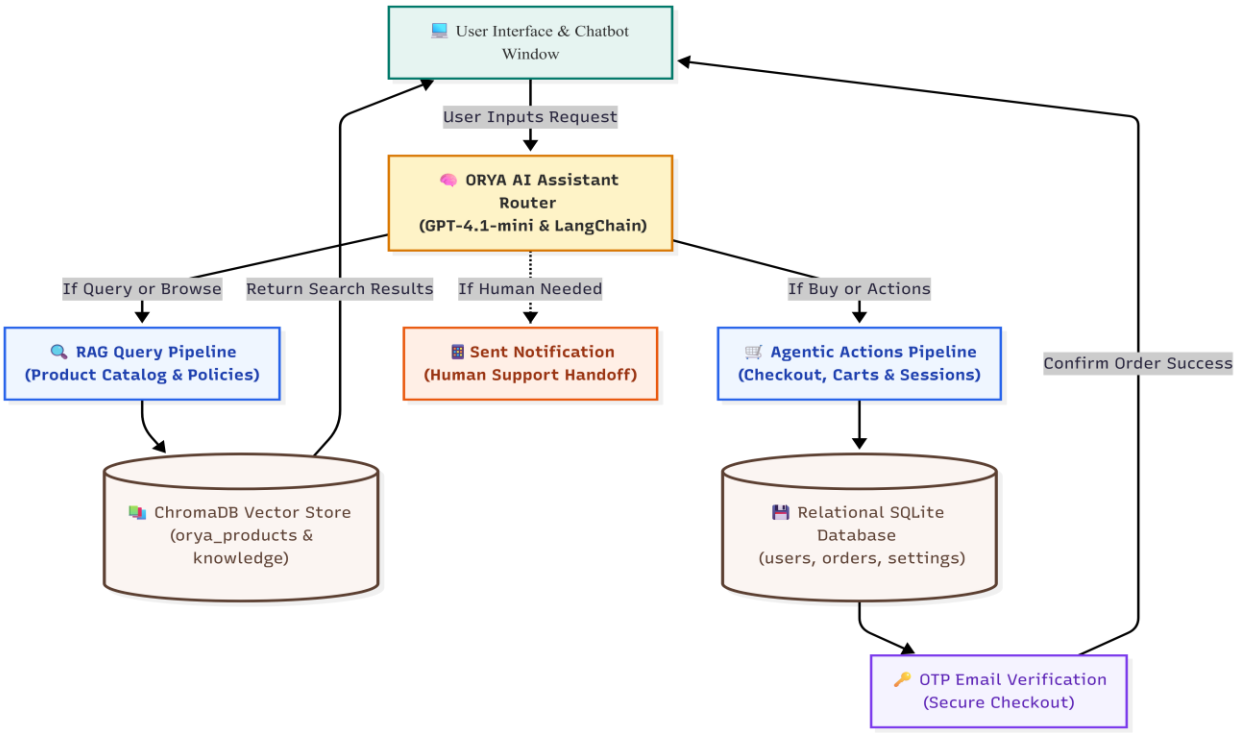


Figure 0.2: Chatbot Routing

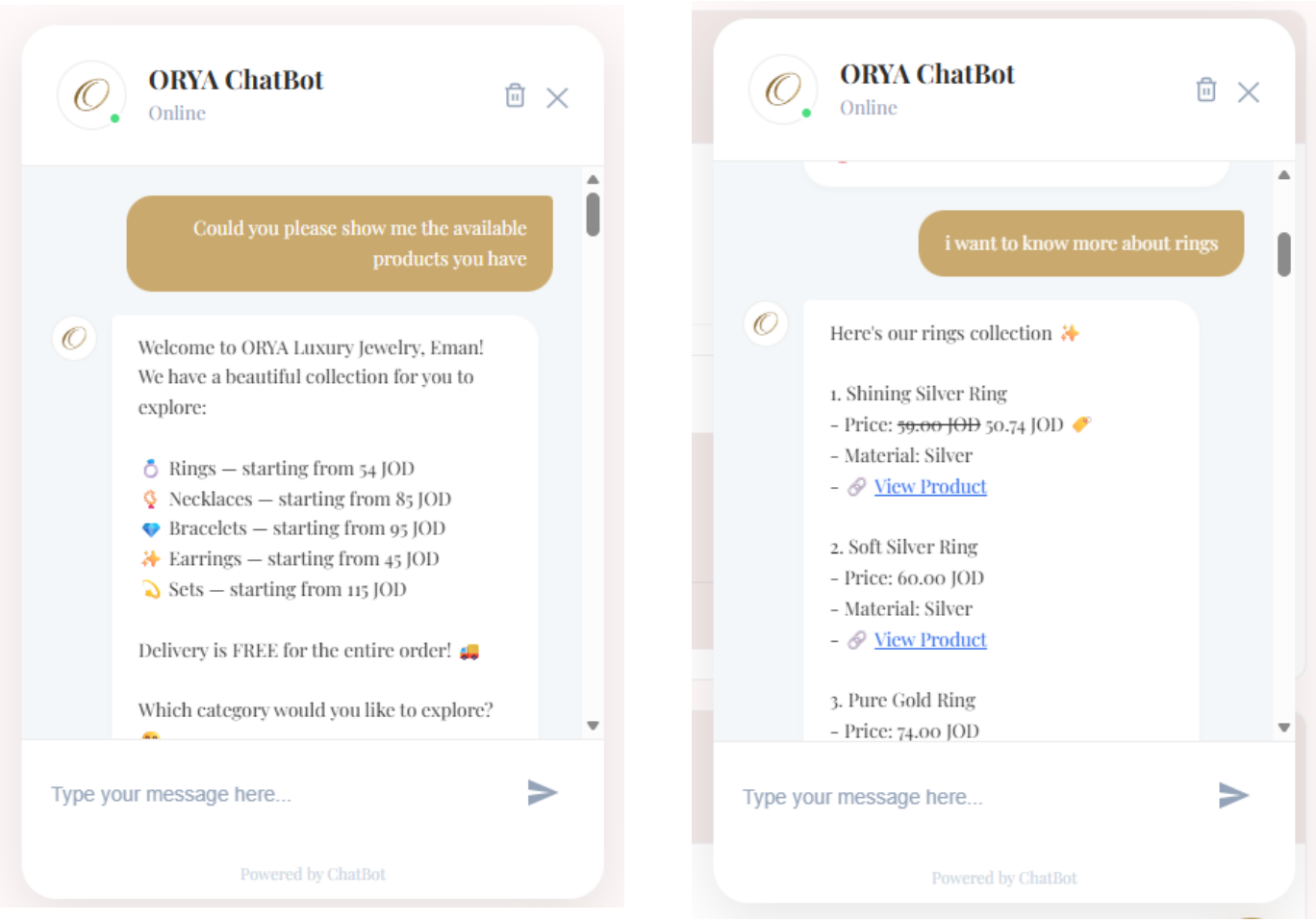


Figure 6.3 : Search Products Tool

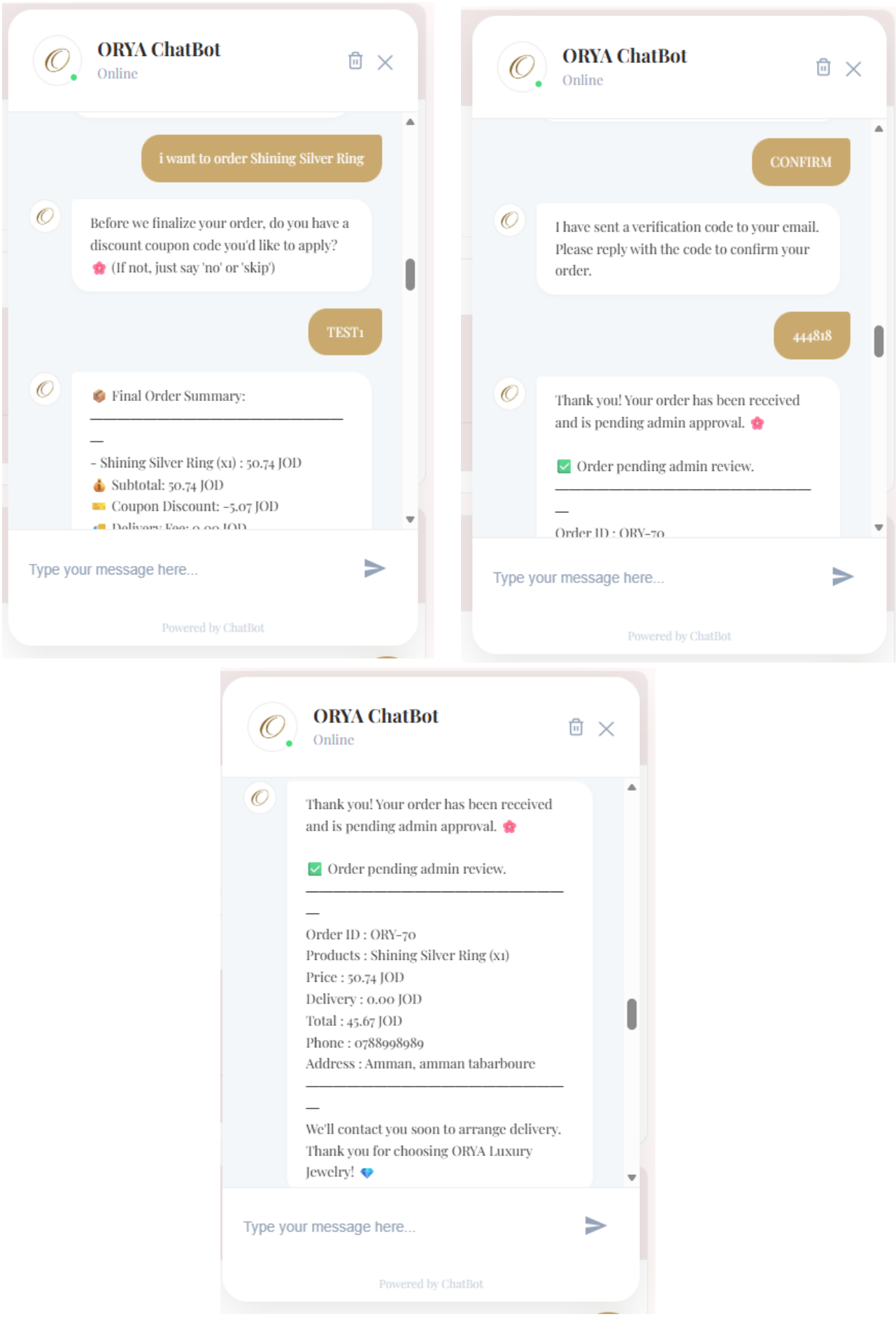


Figure 6.4: Create Purchase Request and OTP Tool

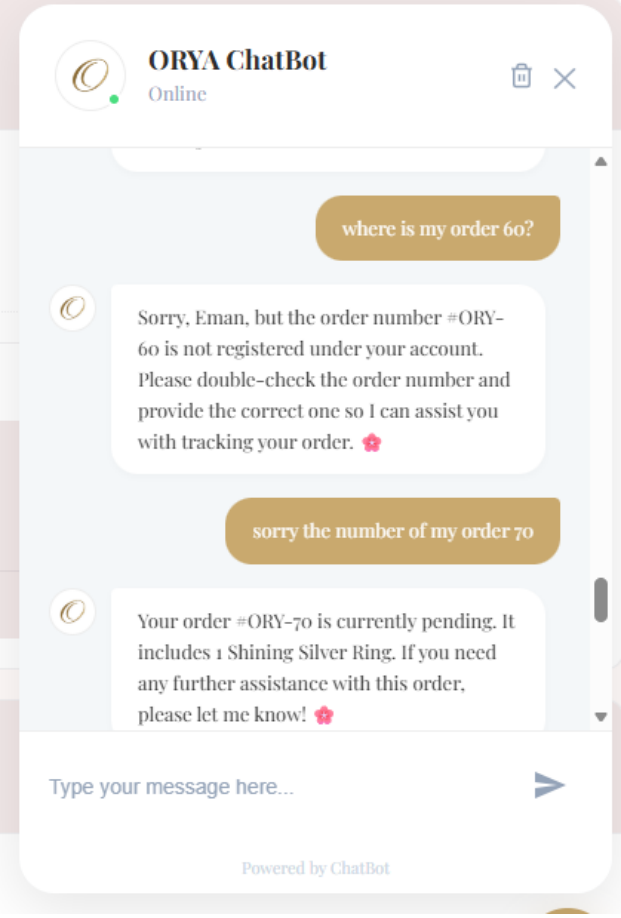


Figure 6.5: Get Order Status Tool

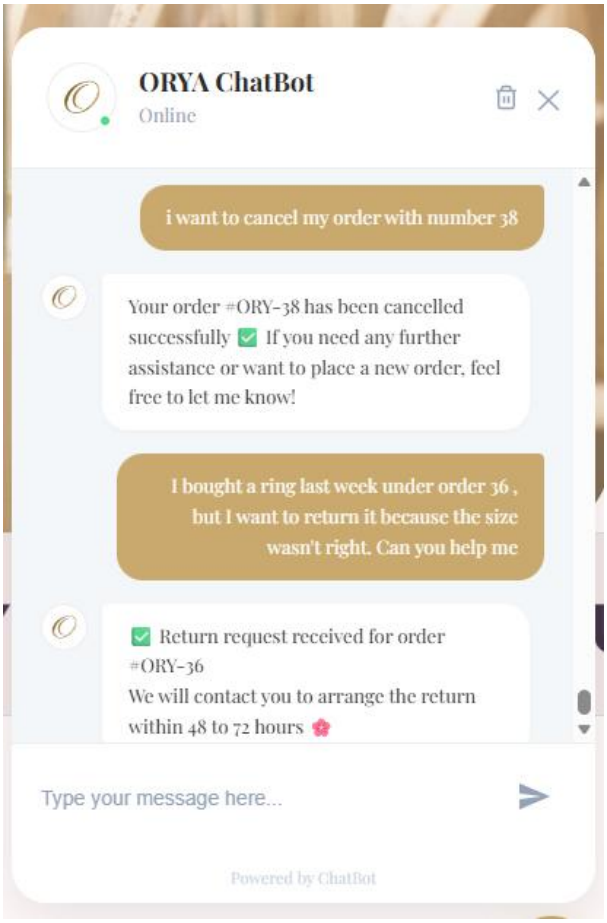


Figure 6.6: Cancel Order and Human Handoff Tool

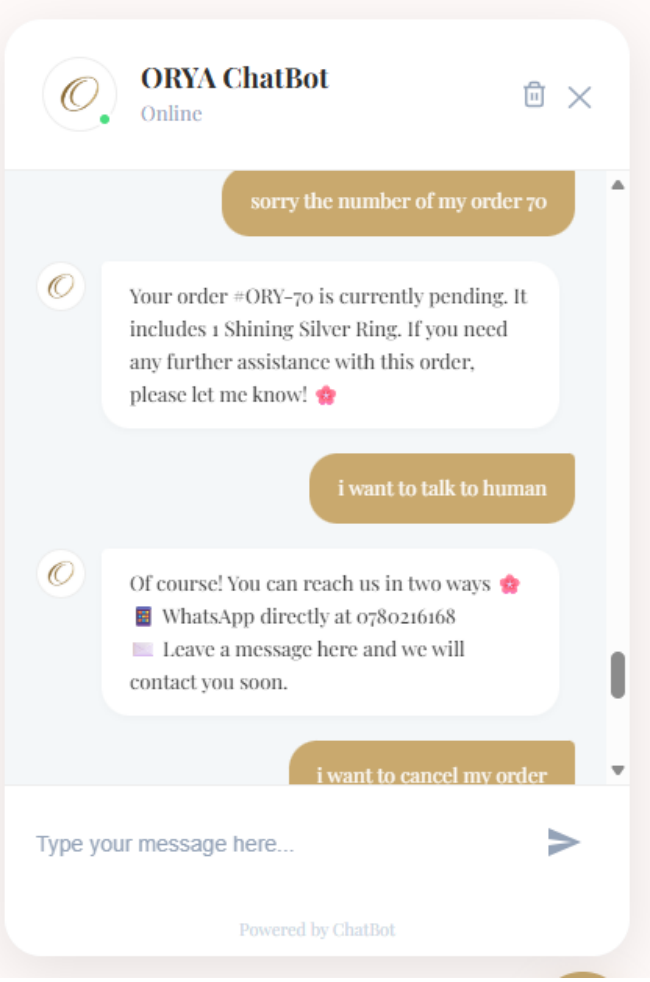


Figure 6.7: Request Return Tool

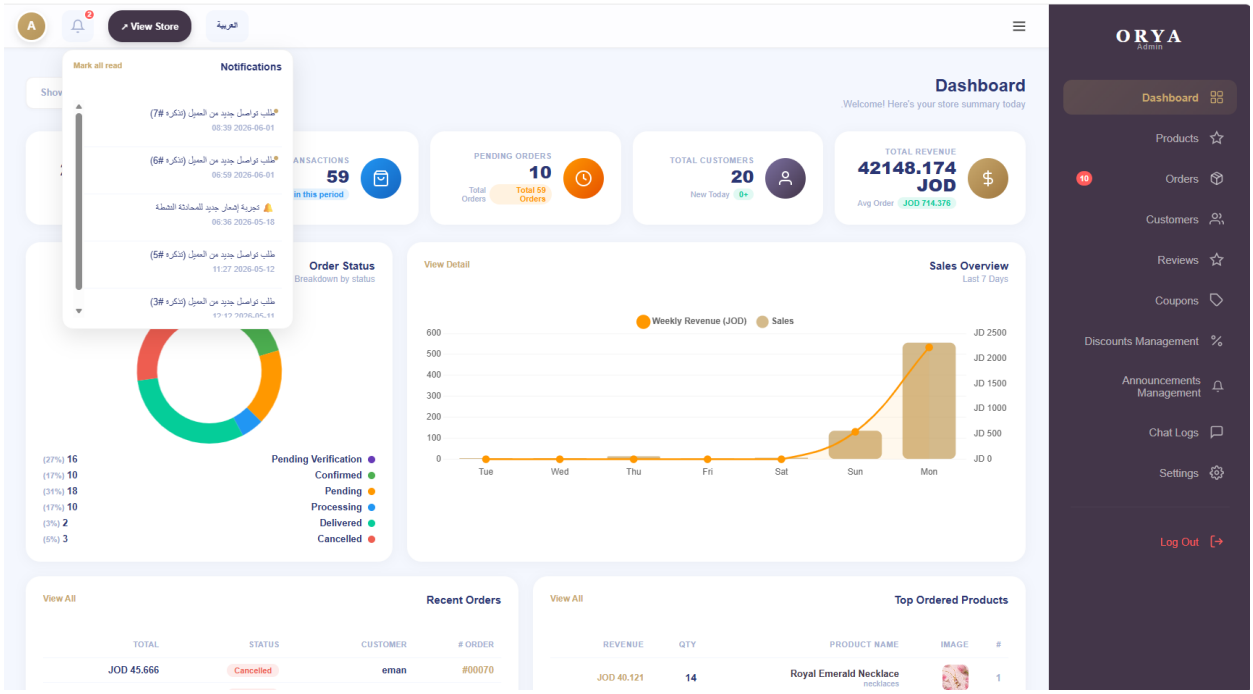


Figure 6.8: Admin Notification Triggered Instantly via the Human Handoff Tool